

INTRODUCTION TO ALGORITHMS AND DATA STRUCTURES

Lecture I for course in
Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Exercices

- Show how to implement a queue using two stacks.

Enqueue pushes into “in-stack”
Dequeue pops elements from “out-stack”.

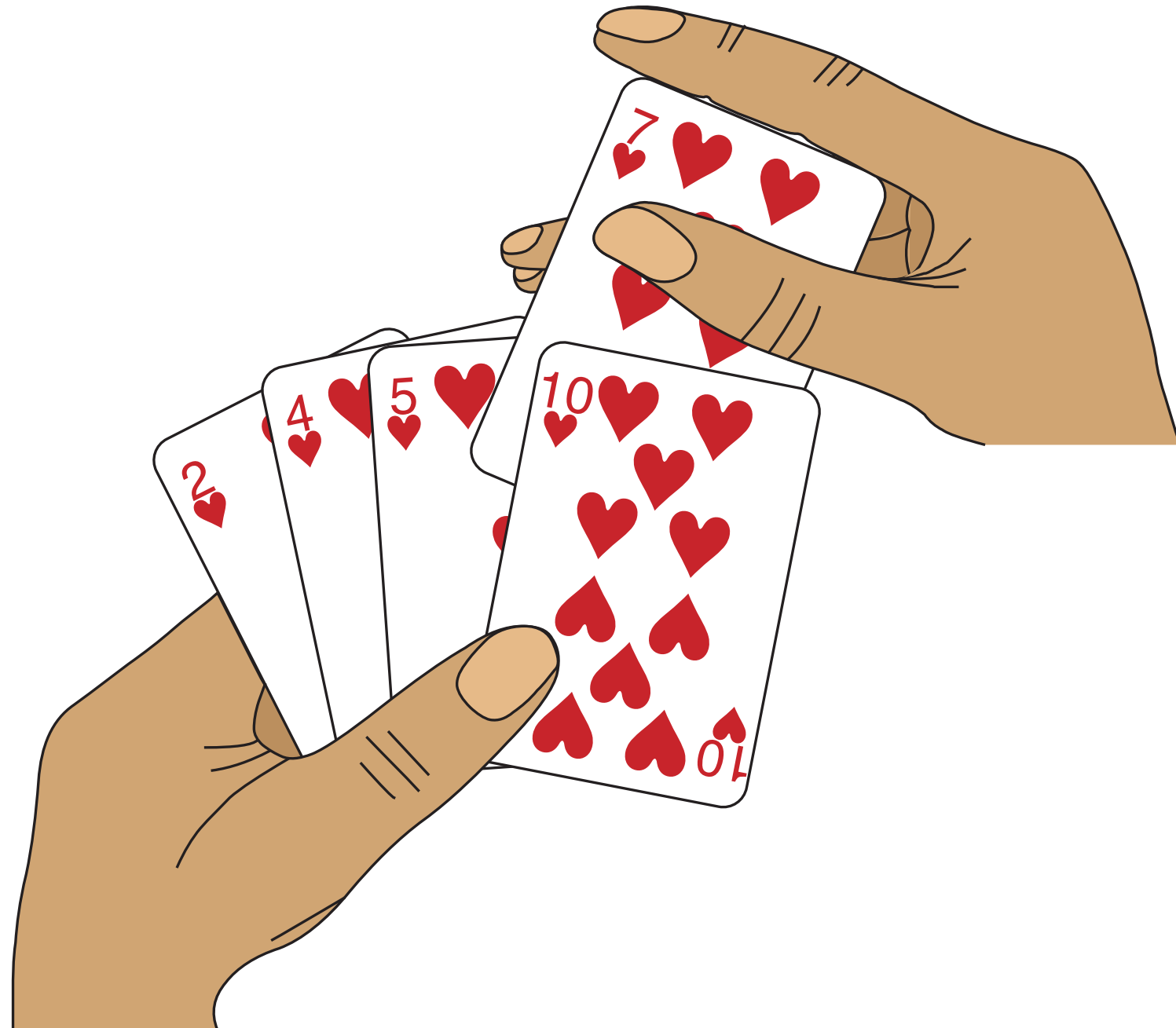
Unless “out-stack” is empty, in which case:
it Pops everything from “in-stack” and Pushes into “out-stack” first.

ALGORITHM ANALYSIS AND ELEMENTARY SORTING ALGORITHMS

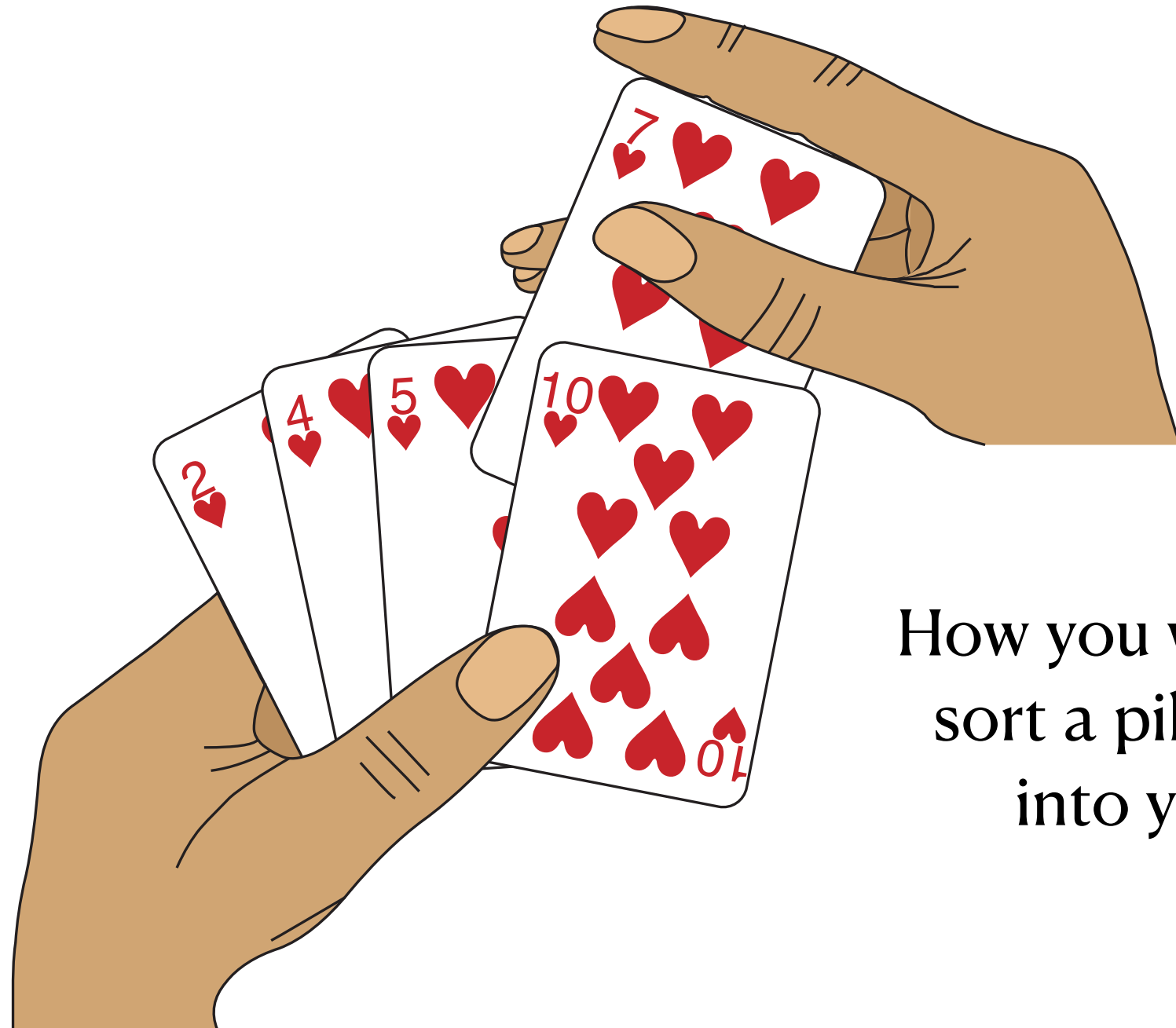
Lecture II for course in
Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Insertion Sort



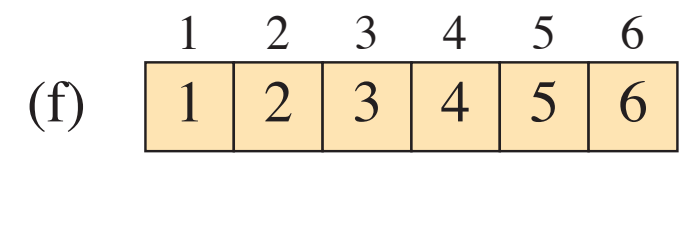
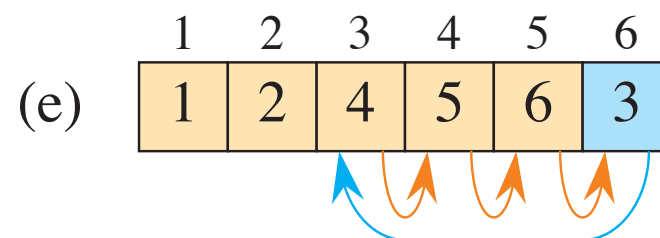
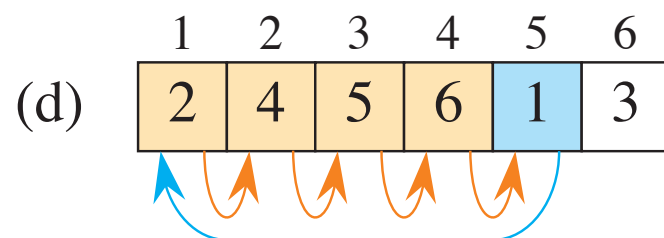
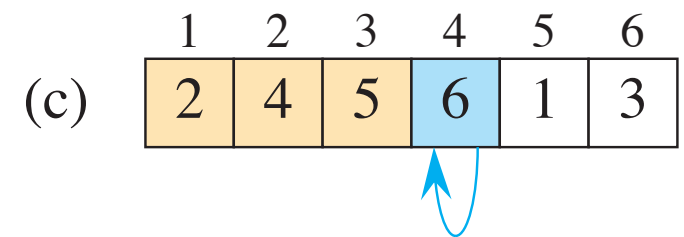
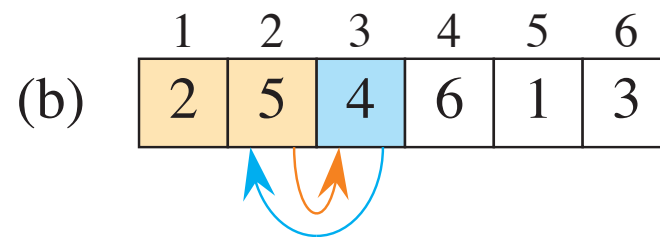
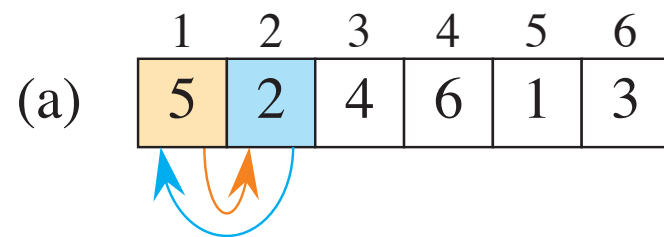
Insertion Sort



How would you
sort a pile of cards
into your hand?

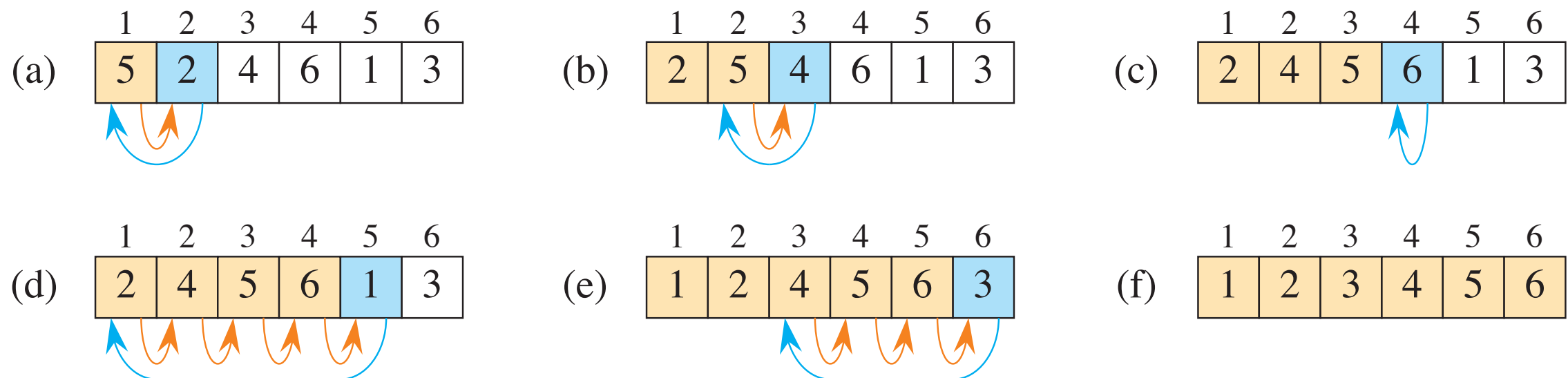
Insertion Sort

- Given an array of un-sorted elements, begin by comparing the second element to the first (a), the third to the first two (b) ...



Insertion Sort

- Given an array of un-sorted elements, begin by comparing the second element to the first (a), the third to the first two (b) ...



- When preparing to compare element i , elements 1 through $i-1$ have already been sorted.
- We continue as such until all elements have been sorted.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

```
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```


Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

The index i indicates the element we will be comparing.

```
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$

The index i indicates the
element we will be comparing.

Hence called *key*.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$

We write *//* to denote
a line is a comment

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$

until we reach a
value smaller than the key.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$ We move the element

6 $A[j + 1] = A[j]$ before the key forward,

7 $j = j - 1$ until we reach a

8 $A[j + 1] = key$ value smaller than the key.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$ We move the element

6 $A[j + 1] = A[j]$ before the key forward, and

7 $j = j - 1$ keep shifting, until we reach a

8 $A[j + 1] = key$ value smaller than the key.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

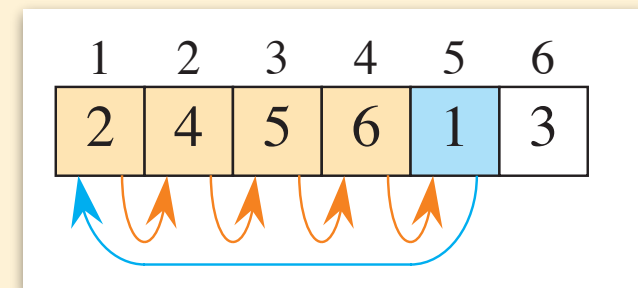
4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$



Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$

Finally, we place the
key, in the gap
following the first
element we found
smaller than the key.

Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

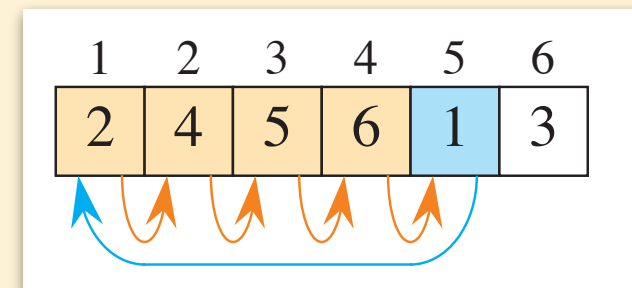
4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$



Insertion Sort

Consider an array A , with n values (**keys**), to be sorted.

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

2 $key = A[i]$

3 *// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.*

4 $j = i - 1$

5 **while** $j > 0$ and $A[j] > key$

6 $A[j + 1] = A[j]$

7 $j = j - 1$

8 $A[j + 1] = key$

As $A[1 : i - 1]$ is
sorted at the
beginning of each
iteration of the for
loop, we say it is a
loop invariant.

Loop invariants

- To prove a statement is a loop invariant, we must show:
 - it holds true prior to the first iteration of the loop;

Loop invariants

- To prove a statement is a loop invariant, we must show:

Loop invariants

- To prove a statement is a loop invariant, we must show:
 - it holds true prior to the first iteration of the loop;
 - if it is true before an iteration, it remains true before the next iteration;

Loop invariants

- To prove a statement is a loop invariant, we must show:
 - it holds true prior to the first iteration of the loop;
 - if it is true before an iteration, it remains true before the next iteration;
 - upon termination of the loop, the invariant (along with the condition which caused the loop to terminate) help to determine the algorithm is correct.

Loop invariants

- To prove a statement is a loop invariant, we must show:
 - it holds true prior to the first iteration of the loop;
 - if it is true before an iteration, it remains true before the next iteration;
 - upon termination of the loop, the invariant (along with the condition which caused the loop to terminate) help to determine the algorithm is correct.
- We refer to the three properties as **Initialization**, **Maintenance** and **Termination** respectively.

Insertion Sort

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

- **Initialization:** let us show $A[1 : i - 1]$ is sorted before the first loop iteration.

Take $i=2$, $A[1 : i - 1] = A[1]$

Insertion Sort

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

- **Initialization:** let us show $A[1 : i - 1]$ is sorted before the first loop iteration.

Take $i=2$, $A[1 : i - 1] = A[1]$

An array with a single element is sorted without question.

The loop invariant thus holds before the first loop iteration.

Insertion Sort

```
4       $j = i - 1$   
5      while  $j > 0$  and  $A[j] > key$   
6           $A[j + 1] = A[j]$   
7           $j = j - 1$   
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Insertion Sort

```
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Notice the iteration splits $A[1 : i]$ into three subarrays:

$$A[1 : j], A[j + 1], A[j + 2 : i]$$

Where j is such that and $A[j] < key = A[j + 1] < A[j + 2]$

Insertion Sort

```
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Notice the iteration splits $A[1 : i]$ into three subarrays:

$A[1 : j]$, $A[j + 1]$, $A[j + 2 : i]$

- $A[1 : j]$ remains untouched, thus sorted by hypothesis

Insertion Sort

```
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Notice the iteration splits $A[1 : i]$ into three subarrays:

$A[1 : j]$, $A[j + 1]$, $A[j + 2 : i]$

- $A[j + 1]$ is undeniably sorted

Insertion Sort

```
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Notice the iteration splits $A[1 : i]$ into three subarrays:

$$A[1 : j], A[j + 1], A[j + 2 : i]$$

- $A[j + 2 : i]$ holds the values, shifted by one, of $A[j + 1 : i - 1]$, so it is sorted by hypothesis

Insertion Sort

```
4       $j = i - 1$   
5      while  $j > 0$  and  $A[j] > key$   
6           $A[j + 1] = A[j]$   
7           $j = j - 1$   
8       $A[j + 1] = key$ 
```

- **Maintenance:** let us show $A[1 : i]$ is sorted after a loop iteration, if $A[1 : i - 1]$ was sorted before the iteration.

Notice the iteration splits $A[1 : i]$ into three subarrays:

$$A[1 : j], A[j + 1], A[j + 2 : i]$$

Each subarray being sorted and having, $A[j] < A[j + 1] < A[j + 2]$, we conclude the subarray $A[1 : i]$ is sorted after the iteration.

Insertion Sort

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

- **Termination:** let us show the algorithm is correct, using the loop invariant, $A[1 : i - 1]$ is sorted before iteration i .

Insertion Sort

INSERTION-SORT(A, n)

1 **for** $i = 2$ **to** n

- **Termination:** let us show the algorithm is correct, using the loop invariant, $A[1 : i - 1]$ is sorted before iteration i .

When the loop terminates, $i=n+1$, thus $A[1 : i - 1] = A[1 : n]$ is sorted.

This immediately concludes that the algorithm is correct.

Exercices

- Find a loop invariant and use it to show the algorithm is correct.

SUM-ARRAY(A, n)

1 $sum = 0$

2 **for** $i = 1$ **to** n

3 $sum = sum + A[i]$

4 **return** sum

Exercices

- Find a loop invariant and use it to show the algorithm is correct.

SUM-ARRAY(A, n)

1 $sum = 0$

2 **for** $i = 1$ **to** n

3 $sum = sum + A[i]$

4 **return** sum

before the i 'th iteration, sum contains the sum of $A[0 : i-1]$

Homework (due with Assignment 1)

- Write an algorithm for a linear search of array $A[1:n]$ looking for element x .

The algorithm should scan through the array, starting at the first element, and return the index of element x , or the special value NIL if the element is not found.

Describe a loop invariant, and use it to show your algorithm is correct.

Running time analysis

- Denote by c_k the cost of running line k .
- Let t_i be the number of times the **while** loop is run, for given i .

INSERTION-SORT(A, n)		<i>cost</i>	<i>times</i>
1	for $i = 2$ to n	c_1	n
2	$key = A[i]$	c_2	$n - 1$
3	<i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

- Notice the test of the loop is executed one more time than the loop body.

Running time analysis

```
5      while  $j > 0$  and  $A[j] > key$ 
```

- In the best case scenario (fastest), the array is initially sorted and the **while** loop exits immediately, so $t_i = 1$ for any i .

Running time analysis

```
5      while  $j > 0$  and  $A[j] > key$ 
```

- In the best case scenario (fastest), the array is initially sorted and the **while** loop exits immediately, so $t_i = 1$ for any i .
- In the worst case scenario, the array is in reverse order, so the key must be compared with each element of the sorted subarray, $A[1 : i-1]$, thus $t_i = \text{size of sorted subarray} = i$.

Running time analysis

```
5      while  $j > 0$  and  $A[j] > key$ 
```

- In the best case scenario (fastest), the array is initially sorted and the **while** loop exits immediately, so $t_i = 1$ for any i .
- In the worst case scenario, the array is in reverse order, so the key must be compared with each element of the sorted subarray, $A[1 : i-1]$, thus $t_i = \text{size of sorted subarray} = i$.
- In an “average case”, half the elements of the sorted subarray are larger than the key, so $t_i \sim i/2$, roughly as bad as the worst case scenario.

Worse case scenario analysis

INSERTION-SORT(A, n)		<i>cost</i>	<i>times</i>
1	for $i = 2$ to n	c_1	n
2	$key = A[i]$	c_2	$n - 1$
3	<i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

- Let $T(n)$ be the total running time in the worst case scenario ($t_i = i$).

$$T(n) = c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n t_i + (c_6 + c_7) \sum_{i=2}^n (t_i - 1)$$

Worse case scenario analysis

INSERTION-SORT(A, n)		<i>cost</i>	<i>times</i>
1	for $i = 2$ to n	c_1	n
2	$key = A[i]$	c_2	$n - 1$
3	<i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

- Let $T(n)$ be the total running time in the worst case scenario ($t_i = i$).

$$\begin{aligned}
 T(n) &= c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n t_i + (c_6 + c_7) \sum_{i=2}^n (t_i - 1) \\
 &= c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1)
 \end{aligned}$$

Worse case scenario analysis

- Let $T(n)$ be the total running time in the worst case scenario.

$$T(n) = c_1n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1)$$

Worse case scenario analysis

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1) \\ &= c_1 + (c_1 + c_2 + c_4 + c_8)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (-1) \end{aligned}$$

Worse case scenario analysis

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1) \\ &= c_1 + (c_1 + c_2 + c_4 + c_8)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (-1) \\ &= c_1 + (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i \end{aligned}$$

Worse case scenario analysis

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned}T(n) &= c_1n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1) \\&= c_1 + (c_1 + c_2 + c_4 + c_8)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (-1) \\&= c_1 + (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i \\&= \alpha + \beta n + \theta \sum_{i=2}^n i\end{aligned}$$

substituting, $\alpha = c_1$

$$\beta = (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)$$

$$\theta = c_5 + c_6 + c_7$$

Worse case scenario analysis

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned}T(n) &= c_1 n + (c_2 + c_4 + c_8)(n - 1) + c_5 \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (i - 1) \\&= c_1 + (c_1 + c_2 + c_4 + c_8)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i + (c_6 + c_7) \sum_{i=2}^n (-1) \\&= c_1 + (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)(n - 1) + (c_5 + c_6 + c_7) \sum_{i=2}^n i \\&= \alpha + \beta(n - 1) + \theta \sum_{i=2}^n i \\&= \alpha + \beta(n - 1) + \theta \left(\frac{n(n - 1)}{2} - 1 \right)\end{aligned}$$

substituting, $\alpha = c_1$
 $\beta = (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)$
 $\theta = c_5 + c_6 + c_7$

Order of growth

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned} T(n) &= \alpha + \beta(n - 1) + \theta \left(\frac{n(n - 1)}{2} - 1 \right) \\ &= \frac{\theta}{2}n^2 + \frac{2\beta - \theta}{2}n + (\alpha - \beta - \theta) \end{aligned}$$

Order of growth

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned} T(n) &= \alpha + \beta(n - 1) + \theta \left(\frac{n(n - 1)}{2} - 1 \right) \\ &= \frac{\theta}{2}n^2 + \frac{2\beta - \theta}{2}n + (\alpha - \beta - \theta) \end{aligned}$$

- To further simplify our time running analysis, we focus only on the order of growth, the factor from the leading term: n^2 .
- We say insertion sort has worst case running time of $\Theta(n^2)$.

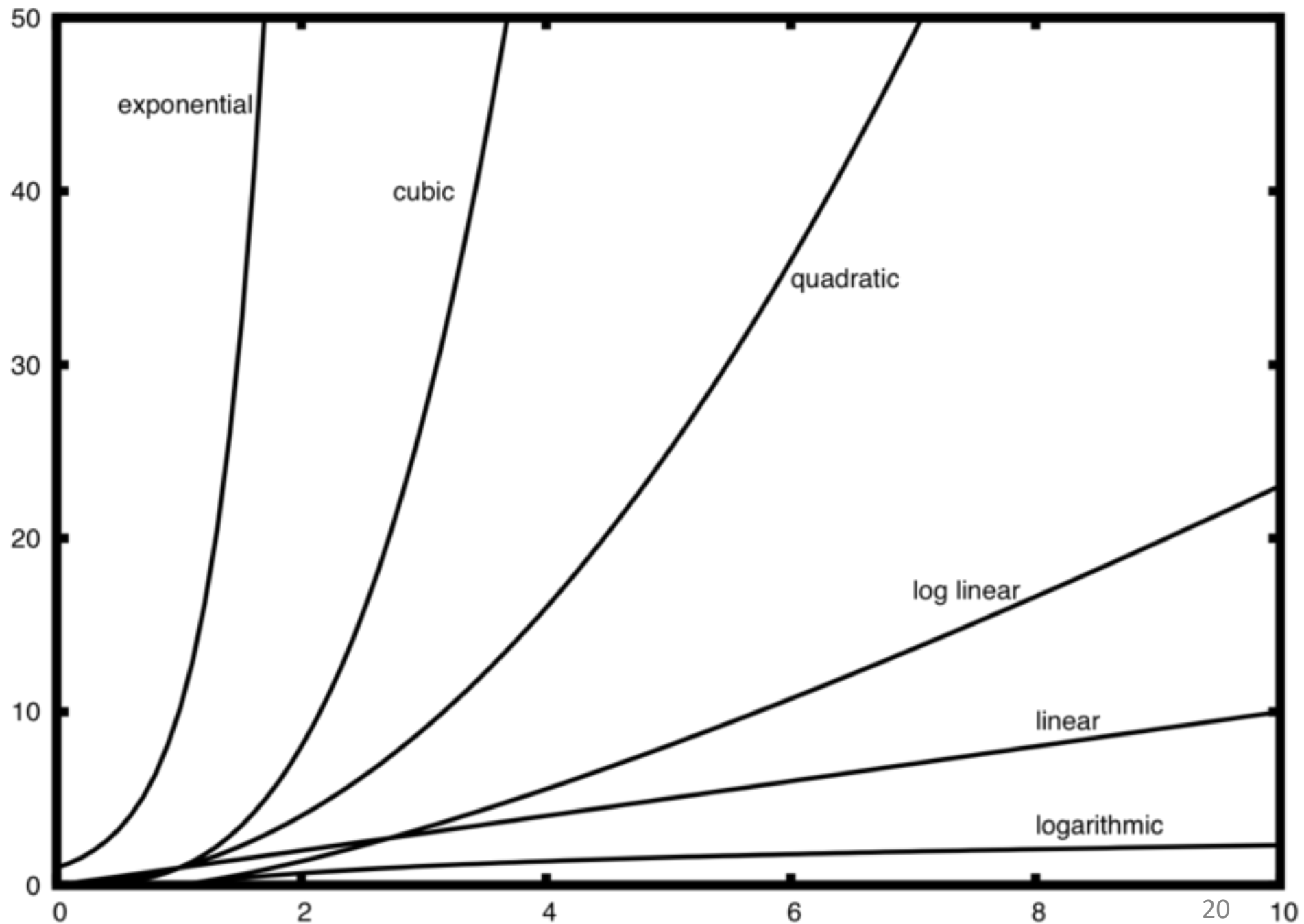
Order of growth

- Let $T(n)$ be the total running time in the worst case scenario.

$$\begin{aligned} T(n) &= \alpha + \beta(n - 1) + \theta \left(\frac{n(n - 1)}{2} - 1 \right) \\ &= \frac{\theta}{2}n^2 + \frac{2\beta - \theta}{2}n + (\alpha - \beta - \theta) \end{aligned}$$

- To further simplify our time running analysis, we focus only on the order of growth, the factor from the leading term: n^2 .
- We say insertion sort has worst case running time of $\Theta(n^2)$.
- This running time analysis is particularly relevant for large enough inputs.

Order of growth



Exercices

Regardless of the constants hidden in Θ - notation, there is always some number say n_0 , such that for any larger input $n \geq n_0$, a $\Theta(n^2)$ algorithm beats an $\Theta(n^3)$ algorithm.

- Say two algorithms have worse case running times:

$$T_1(n) = n^3 + 147n \text{ and } T_2(n) = 21n^2 + 343$$

Find n_0 such that for any $n \geq n_0$, it holds that: $T_1(n) \geq T_2(n)$.

Exercices

Regardless of the constants hidden in Θ - notation, there is always some number say n_0 , such that for any larger input $n \geq n_0$, a $\Theta(n^2)$ algorithm beats an $\Theta(n^3)$ algorithm.

- Say two algorithms have worse case running times:

$$T_1(n) = n^3 + 147n \text{ and } T_2(n) = 21n^2 + 343$$

Find n_0 such that for any $n \geq n_0$, it holds that: $T_1(n) \geq T_2(n)$.

equivalent to finding n_0 s.t. $(n - 7)^3 \geq 0$

Homework (due with Assignment 1)

- Design a sorting algorithm (selection sort)which follows the following procedure. First find the smallest element of $A[1:n]$ and exchange it with the element in $A[1]$. Then find the smallest element of $A[2:n]$, and exchange it with $A[2]$. Continue in this manner for the first $n-1$ elements of A .

What loop invariant does this algorithm maintain?

Give the worst-case running time of selection sort in Θ - notation.

Compare with the best scenario running time.

Θ notation and Big-O notation

- Θ - notation characterizes the *order of growth*

Θ notation and Big-O notation

- Θ - notation characterizes the *order of growth* of the *worst case* running time for an algorithm.

Θ notation and Big-O notation

- Θ - notation characterizes a *tight bound* on the *order of growth* of the *worst case* running time for an algorithm.

Θ notation and Big-O notation

- Θ - notation characterizes a *tight bound* on the *order of growth* of the *worst case* running time for an algorithm.
- Θ - notation is only one of several *asymptotic notations* (focus on the highest order term of the running time).

Θ notation and Big-O notation

- Θ - notation characterizes a *tight bound* on the *order of growth* of the *worst case* running time for an algorithm.
- Θ - notation is only one of several *asymptotic notations* (focus on the highest order term of the running time). The others are:

Big-O notation and Ω notation.

Θ notation and Big-O notation

- Θ - notation characterizes a *tight bound* on the *order of growth* of the *worst case* running time for an algorithm.
- Θ - notation is only one of several *asymptotic notations* (focus on the highest order term of the running time). The others are:

Big-O notation and Ω notation.

- Big-O notation gives an upper bound of the asymptotic behavior.
- Ω notation gives a lower bound.
- If a function is both, say $O(n^3)$ and $\Omega(n^3)$, then it is $\Theta(n^3)$.

Θ notation and Big-O notation

- Θ - notation characterizes a *tight bound* on the *order of growth* of the *worst case* running time for an algorithm.
- Θ - notation is only one of several *asymptotic notations* (focus on the highest order term of the running time). The others are:

Big-O notation and Ω notation.

- Big-O notation gives an upper bound of the asymptotic behavior.
 - Ω notation gives a lower bound.
 - If a function is both, $O(f(n))$ and $\Omega(f(n))$, then it is $\Theta(f(n))$.
- In practice, Big-O notation is used interchangeably with Θ -notation.

Algorithm design

Algorithm design

- We say insertion sort, is an **incremental algorithm**.
- At each step, it increases the size of the currently sorted subarray $A[1:i-1]$ by one, by appropriately placing the element $A[i]$.

Algorithm design

- We say insertion sort, is an **incremental algorithm**.
- At each step, it increases the size of the currently sorted subarray $A[1:i-1]$ by one, by appropriately placing the element $A[i]$.
- Notice the algorithm always makes use of the same action: placing one element in its appropriate place within a sorted list.

Algorithm design

- We say insertion sort, is an **incremental algorithm**.
- At each step, it increases the size of the currently sorted subarray $A[1:i-1]$ by one, by appropriately placing the element $A[i]$.
- Notice the algorithm always makes use of the same action: placing one element in its appropriate place within a sorted list.
- Another useful algorithm design involves a **recursive structure**: To solve a problem, the algorithm calls itself (recurses) to solve related subproblems.

Algorithm design

- We say insertion sort, is an **incremental algorithm**.
- At each step, it increases the size of the currently sorted subarray $A[1:i-1]$ by one, by appropriately placing the element $A[i]$.
- Notice the algorithm always makes use of the same action: placing one element in its appropriate place within a sorted list.
- Another useful algorithm design involves a **recursive structure**: To solve a problem, the algorithm calls itself (recurses) to solve related subproblems.
- These algorithms typically **divide and conquer**: breaking the problem into several smaller subproblems, and then combining.

Divide and conquer algorithms

- The divide and conquer method relies on three steps:
 - **Divide** the problem into one or more subproblems which are smaller instances of the original problem.
 - **Conquer** the subproblems by solving them recursively
 - **Combine** the the subproblem solution to form a solution to the original problem

Divide and conquer algorithms

- The divide and conquer method relies on three steps:
 - **Divide** the problem into one or more subproblems which are smaller instances of the original problem.
 - **Conquer** the subproblems by solving them recursively
 - **Combine** the the subproblem solution to form a solution to the original problem
- One good example is **merge sort** which:
 - splits an array into two subarrays down the middle,
 - recurses down to smaller and smaller arrays,
 - and finally combines the adjacent sorted subarrays, into one singular sorted array.

Merge sort

p			q				r
1	2	3	4	5	6	7	8
12	3	7	9	14	6	11	2

- One good example is **merge sort** which:
 - splits an array into two subarrays down the middle,
 - recurses down to smaller and smaller arrays,
 - and finally combines the adjacent sorted subarrays, into one singular sorted array.

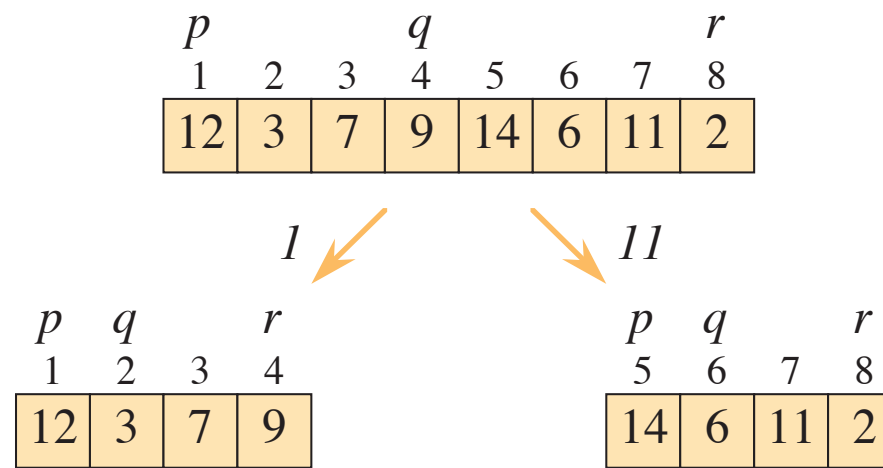
Merge sort

p			q				r
1	2	3	4	5	6	7	8
12	3	7	9	14	6	11	2

- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays, into one singular sorted array.

Merge sort

divide

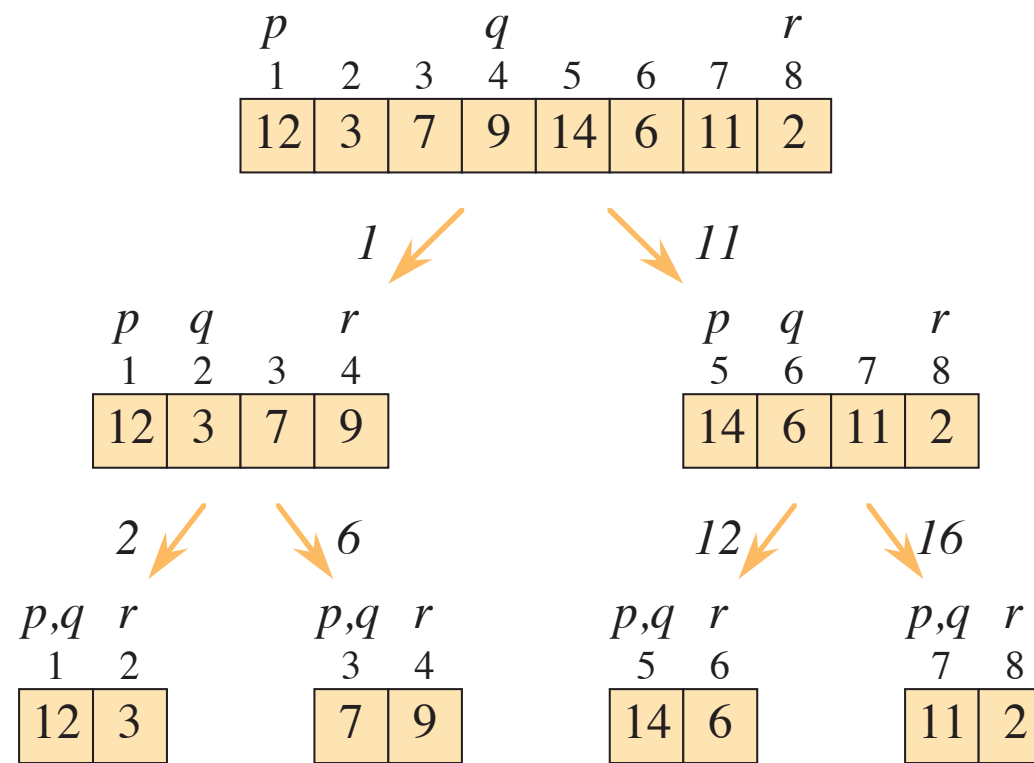


- splits an array into two subarrays down the middle,

Merge sort

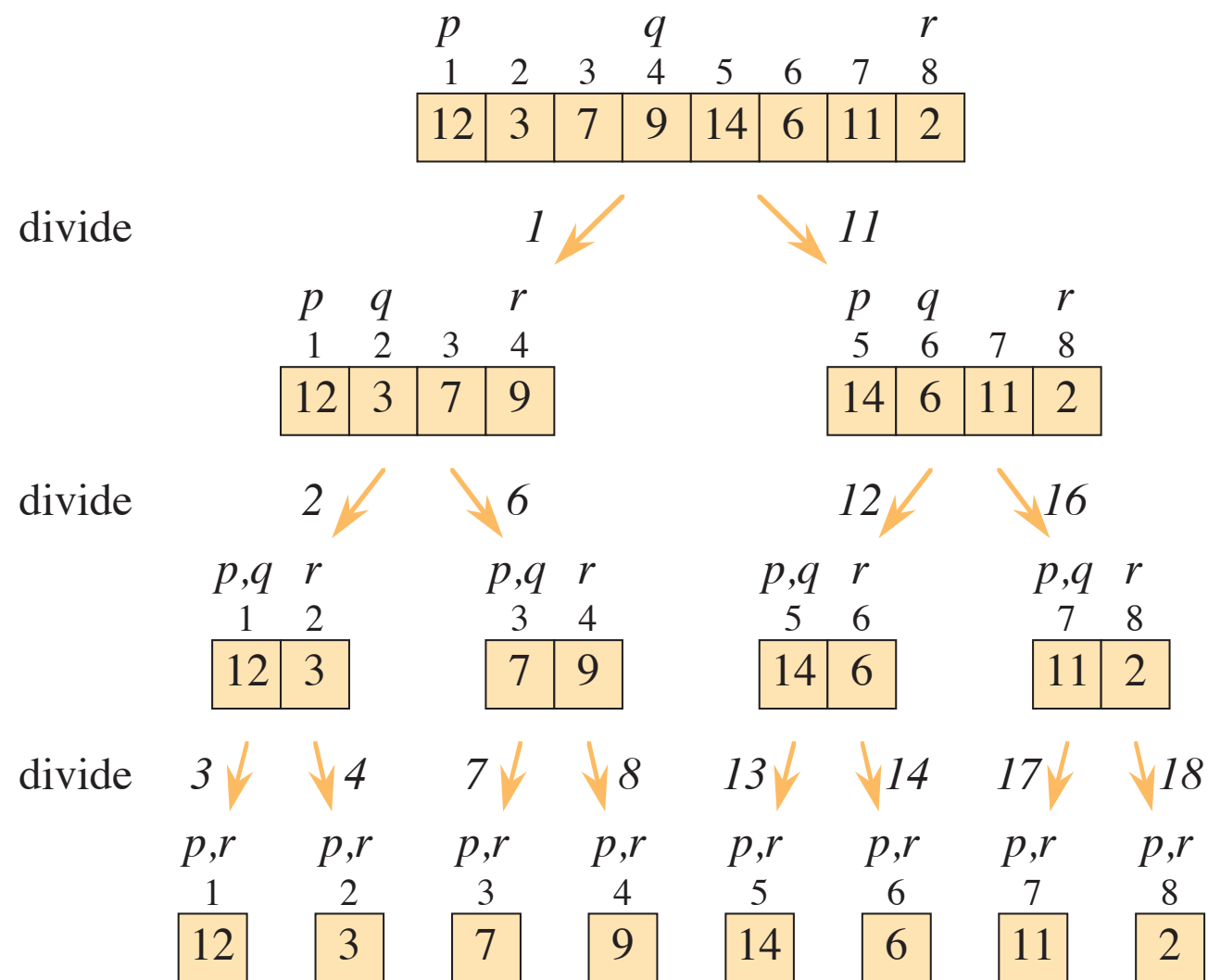
divide

divide



- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,

Merge sort



- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,

Merge sort

divide

divide

divide

merge

p			q				r
1	2	3	4	5	6	7	8
12	3	7	9	14	6	11	2

1

11

p	q		r
1	2	3	4
12	3	7	9

p	q		r
5	6	7	8
14	6	11	2

p, q	r
1	2
12	3

p, q	r
3	4
7	9

p, q	r
5	6
14	6

p, q	r
7	8
11	2

p, r	p, r
1	2
12	3

p, r	p, r
3	4
7	9

p, r	p, r
5	6
14	6

p, r	p, r
7	8
11	2

p, q	r
1	2
3	12

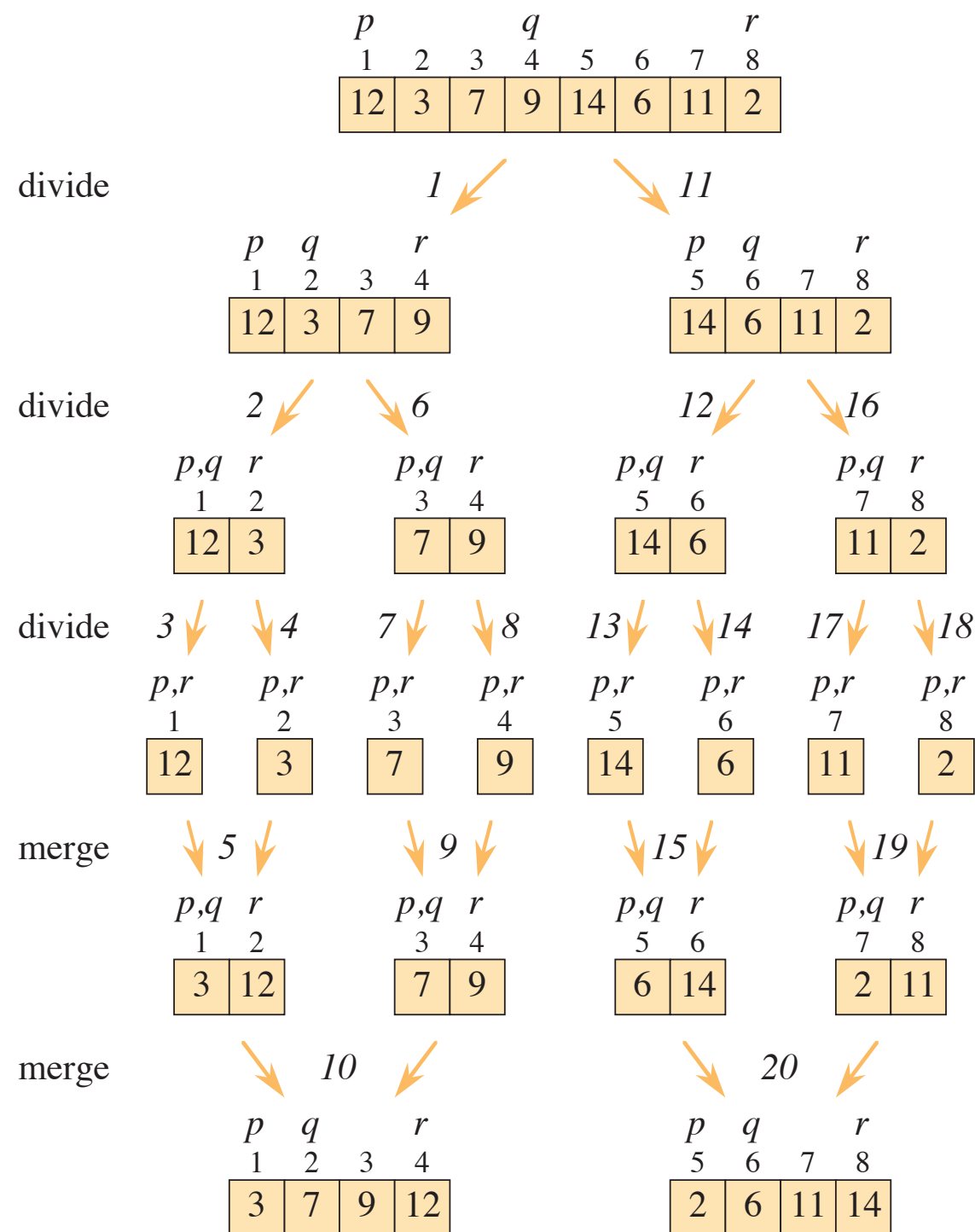
p, q	r
3	4
7	9

p, q	r
5	6
6	14

p, q	r
7	8
2	11

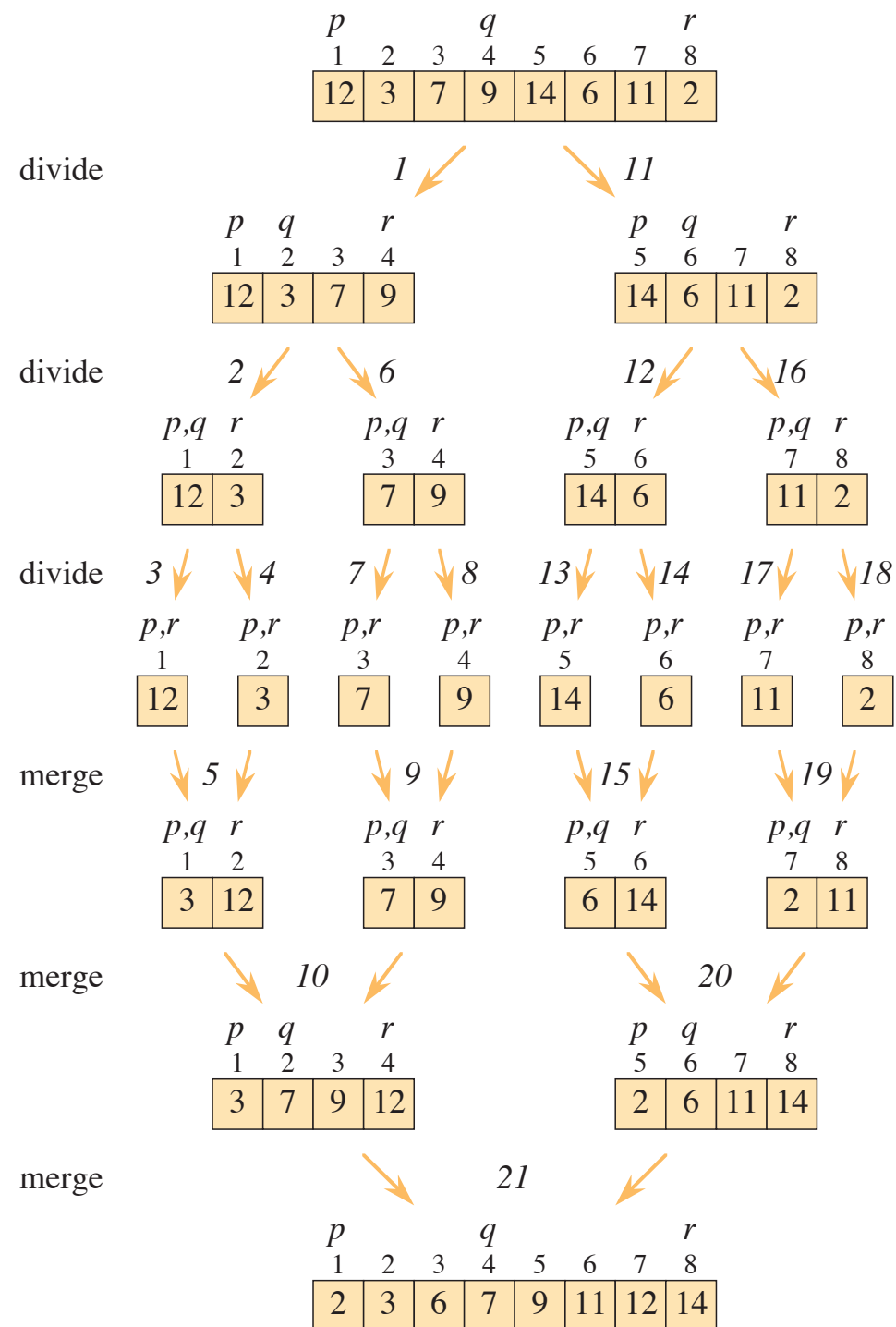
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays

Merge sort



- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays

Merge sort



- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,
- and finally combines the adjacent sorted subarrays, into one singular sorted array.

Merge sort

- Consider an array A , with entries at indexes $p, \dots, q$ to be sorted

MERGE-SORT(A, p, r)

```
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                          // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                      // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
```

Merge sort

- Consider an array A , with entries at indexes $p, \dots, q$ to be sorted

MERGE-SORT(A, p, r)

```
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                          // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                      // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
```

By allowing the entries of interest to be at indexes $p, \dots, q$ instead of $1, \dots, n$, we can directly call the MERGE-SORT in the recursion

Merge sort

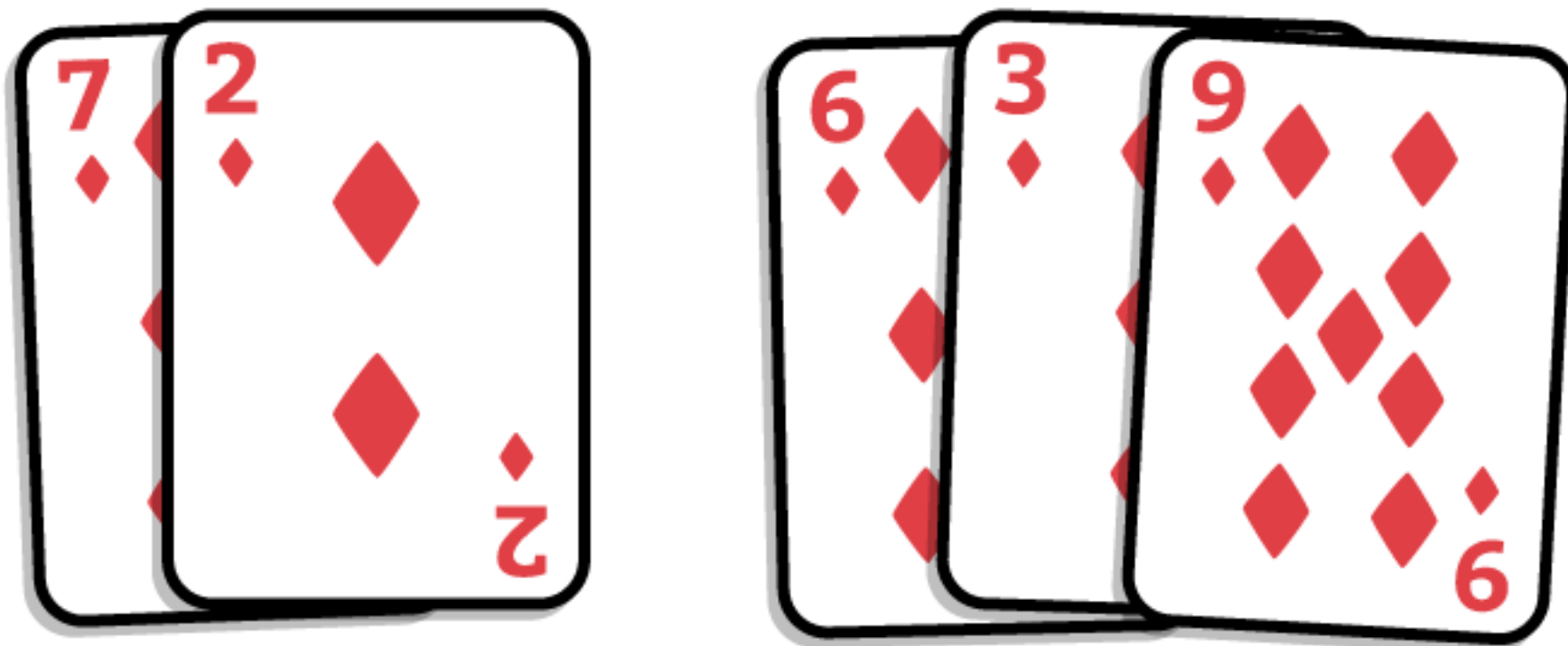
- Consider an array A , with entries at indexes $p, \dots, q$ to be sorted

MERGE-SORT(A, p, r)

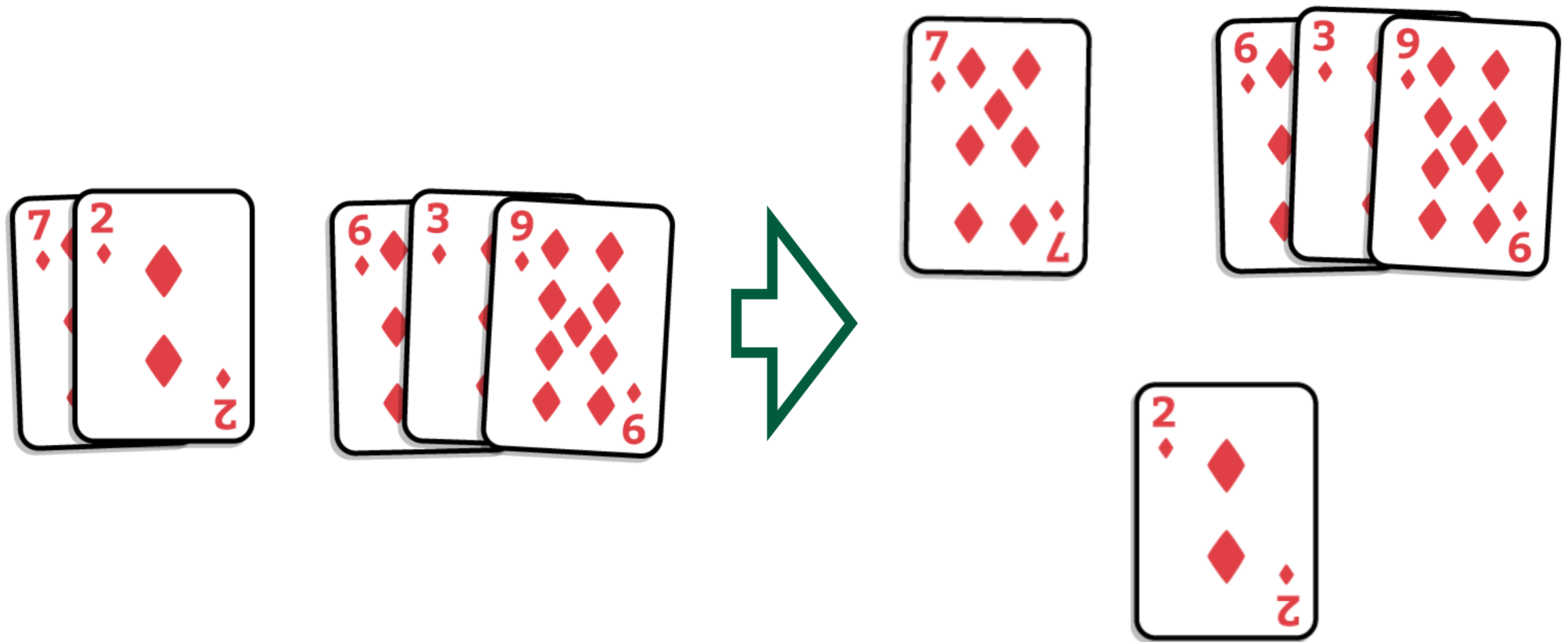
```
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                          // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                      // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
```

The key operation here, is the MERGE procedure.

The MERGE procedure



The MERGE procedure



The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									
					0	1	2	3		
					1	2	3	5		
					j					

Consider the concatenated array A of two sorted arrays L and R .

We wish to fully sort the array, by utilizing the fact that L & R are already sorted.

The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									
	0	1	2	3						
R	1	2	3	5						
	j									

Consider the concatenated array A of two sorted arrays L and R .

We wish to fully sort the array, by utilizing the fact that L & R are already sorted.

To save storage, we will overwrite the values in A .

We indicate with blue the elements of A that have yet to be overwritten.

The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									

	0	1	2	3
R	1	2	3	5
	j			

We overwrite the first element of A with the smallest element of L & R .

Because L & R are sorted, it suffices to compare their first elements to find the smallest.

	8	9	10	11	12	13	14	15	16	17
A	...	1	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									

	0	1	2	3
R	1	2	3	5
	j			

The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									

	0	1	2	3
L	2	4	6	7
	i			

	0	1	2	3
R	1	2	3	5
	j			

	8	9	10	11	12	13	14	15	16	17
A	...	1	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									
	0	1	2	3						
R	1	2	3	5						
	j									

We overwrite the first element of *A* with the smallest element of *L* & *R*.

Because *L* & *R* are sorted, it suffices to compare their first elements to find the smallest.

The first element of *A* is now tan, to indicate it is the final element in its position.

The first element of *R* is now brown, to indicate it has been copied over.

The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
<i>A</i>	...	2	4	6	7	1	2	3	5	...
	<i>k</i>									
	0	1	2	3		0	1	2	3	
<i>L</i>	2	4	6	7		<i>R</i>	1	2	3	5
	<i>i</i>						<i>j</i>			

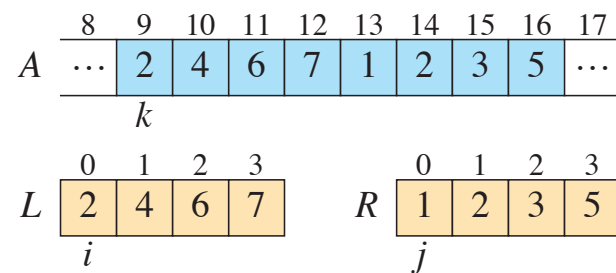
We continue this procedure, now overwriting the second element of *A*.

We now choose the smallest element of *L* & *R*, without considering the element that was already copied over.

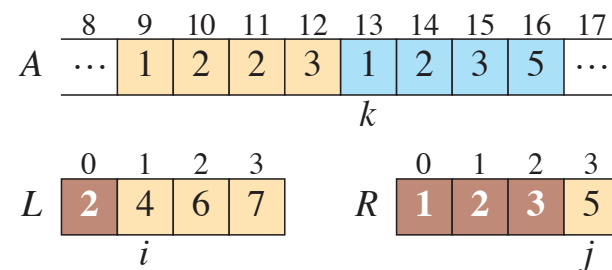
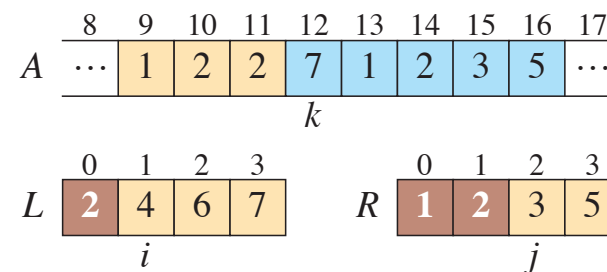
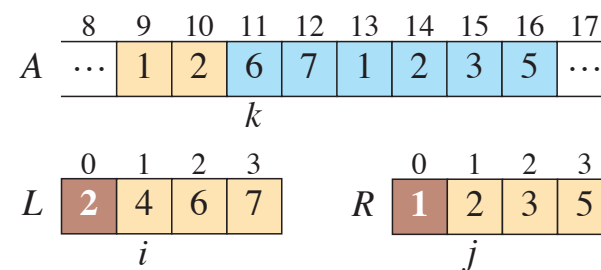
	8	9	10	11	12	13	14	15	16	17
<i>A</i>	...	1	4	6	7	1	2	3	5	...
	<i>k</i>									
	0	1	2	3		0	1	2	3	
<i>L</i>	2	4	6	7		<i>R</i>	1	2	3	5
	<i>i</i>						<i>j</i>			

	8	9	10	11	12	13	14	15	16	17
<i>A</i>	...	1	2	6	7	1	2	3	5	...
	<i>k</i>									
	0	1	2	3		0	1	2	3	
<i>L</i>	2	4	6	7		<i>R</i>	1	2	3	5
	<i>i</i>						<i>j</i>			

The MERGE procedure

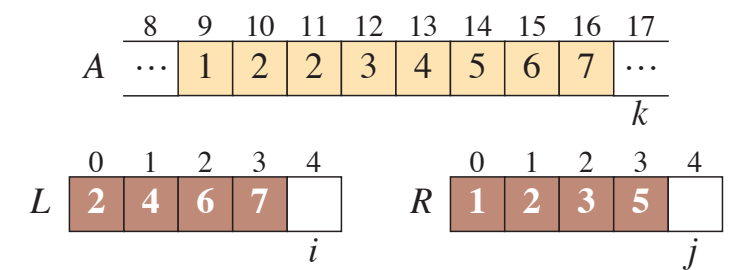


...



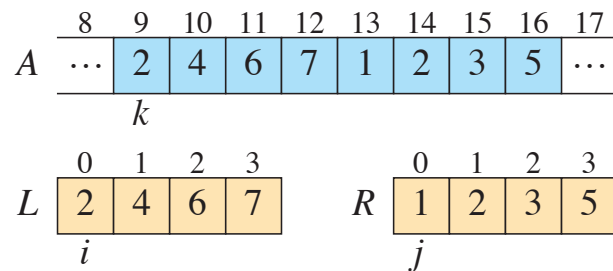
...

We continue so,
until all elements
of A have been
overwritten.

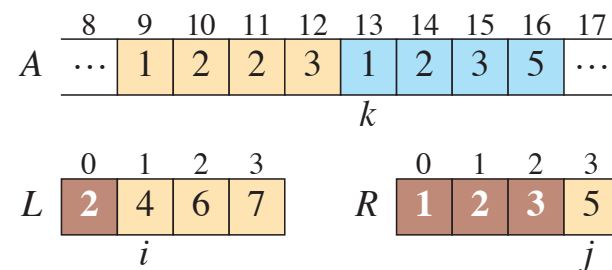
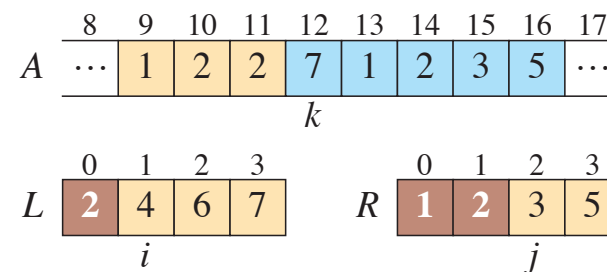
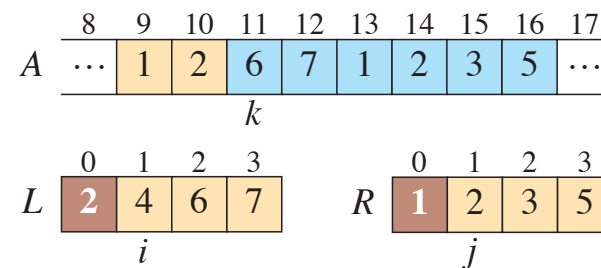


This outputs a sorted array A, corresponding to the merger of sorted arrays L & R.

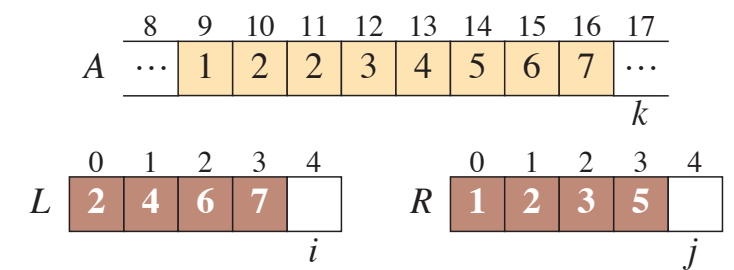
Exercices



...

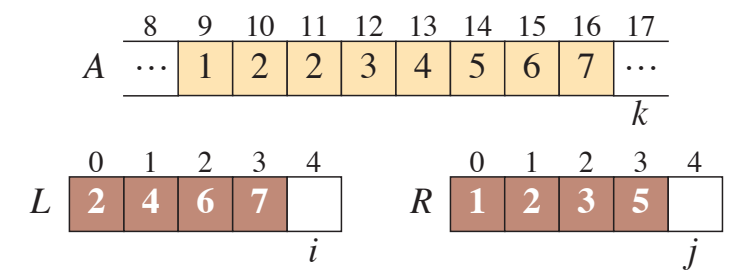
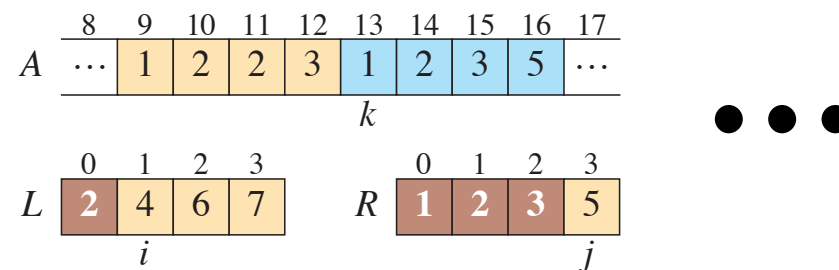
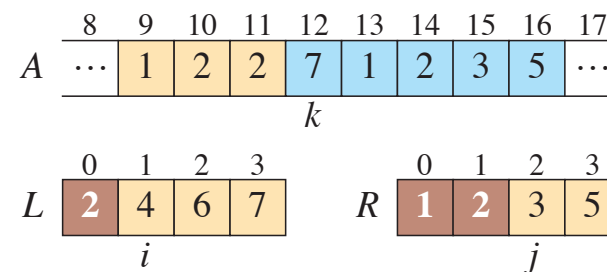
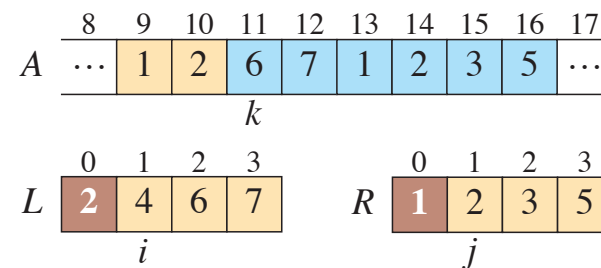
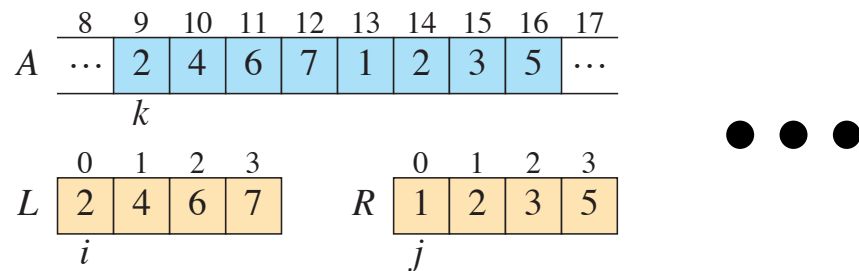


...



Write pseudocode for the MERGE procedure of an array A , which has been split into sorted subarrays, ranging indexes $p, \dots, q$ and $q+1, \dots, r$.

Exercices



The first step should be to create temporary arrays L & R with the entries of $A[p:q]$ and $A[q+1:r]$

Write pseudocode for the MERGE procedure of an array A , which has been split into sorted subarrays, ranging indexes $p, \dots, q$ and $q+1, \dots, r$.

Exercices

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
		k								
L	0	1	2	3						
	2	4	6	7						
	i									

...

	8	9	10	11	12	13	14	15	16	17
A	...	1	2	6	7	1	2	3	5	...
				k						
L	0	1	2	3						
	2	4	6	7						
	i									

	0	1	2	3
R	1	2	3	5
	j			

Define indexes i and j which indicate the smallest remaining elements of L and R .

The first step should be to create temporary arrays L & R with the entries of $A[p:q]$ and $A[q+1:r]$

	8	9	10	11	12	13	14	15	16	17
A	...	1	2	2	7	1	2	3	5	...
				k						
L	0	1	2	3						
	2	4	6	7						
	i									

	0	1	2	3
R	1	2	3	5
	j			

	8	9	10	11	12	13	14	15	16	17
A	...	1	2	2	3	1	2	3	5	...
					k					
L	0	1	2	3						
	2	4	6	7						
	i									

	0	1	2	3
R	1	2	3	5
	j			

...

	8	9	10	11	12	13	14	15	16	17
A	...	1	2	2	3	4	5	6	7	...
					k					
L	0	1	2	3	4					
	2	4	6	7						
	i									

	0	1	2	3	4
R	1	2	3	5	
	j				

Write pseudocode for the MERGE procedure of an array A , which has been split into sorted subarrays, ranging indexes $p, \dots, q$ and $q+1, \dots, r$.

MERGE(A, p, q, r)

```
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                      //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                      //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                      //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //     copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19      $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //     remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```

Exercices

Given pseudocode for the MERGE procedure of an array A , which has been split into sorted subarrays, ranging indexes $p, \dots, q$ and $q+1, \dots, r$.

- Calculate the worst case running time for the MERGE procedure in Θ -notation.

Exercices

Given pseudocode for the MERGE procedure of an array A , which has been split into sorted subarrays, ranging indexes $p, \dots, q$ and $q+1, \dots, r$.

- Calculate the worst case running time for the MERGE procedure in Θ -notation.

$\Theta(n)$ since each basic step takes constant time,
and every value will be copied once into L or R (**for** loops)
and once again back into A (**while** loops)

The MERGE procedure

	8	9	10	11	12	13	14	15	16	17
A	...	2	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									
	0	1	2	3						
R	1	2	3	5						
	j									

	8	9	10	11	12	13	14	15	16	17
A	...	1	4	6	7	1	2	3	5	...
	k									
	0	1	2	3						
L	2	4	6	7						
	i									
	0	1	2	3						
R	1	2	3	5						
	j									

We overwrite the first element of *A* with the smallest element of *L* & *R*.

Because *L* & *R* are sorted, it suffices to compare their first elements to find the smallest.

The first element of *A* is now tan, to indicate it is the final element in its position.

The first element of *R* is now brown, to indicate it has been copied over.

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
 - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
 - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.
 - **Combine** The merge procedure takes $\Theta(n)$ time.

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
 - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.
 - **Combine** The merge procedure takes $\Theta(n)$ time.
- Adding them together, we get

$$T(n) = 2T(n/2) + \Theta(n)$$

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
 - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.
 - **Combine** The merge procedure takes $\Theta(n)$ time.
- Adding them together, we get

$$T(n) = 2T(n/2) + \Theta(n)$$

$$\text{Notice } \Theta(n) + \Theta(1) = \Theta(n)$$

Analysis of merge sort

- We compute the worst case running time of merge sort, $T(n)$, recursively, distinguishing the three stages of the algorithm:
 - **Divide** computing the middle of the array takes constant time $\Theta(1)$.
 - **Conquer** recursively solving two subproblems, each of size $n/2$, takes a total running time of $2T(n/2)$.
 - **Combine** The merge procedure takes $\Theta(n)$ time.
- Adding them together, we get

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

Analysis of merge sort

- The worst case running time of merge sort, $T(n)$ is:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

- To understand this equality, think “How many times did we recursively call the algorithm? ”

Analysis of merge sort

- The worst case running time of merge sort, $T(n)$ is:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

- To understand this equality, think “How many time did we recursively call the algorithm?”
 - At each step, we make the subproblems half the size. So, at step i , the subproblems are each of size $n/(2^i)$.

Analysis of merge sort

- The worst case running time of merge sort, $T(n)$ is:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

- To understand this equality, think “How many time did we recursively call the algorithm?”
 - At each step, we make the subproblems half the size. So, at step i , the subproblems are each of size $n/(2^i)$.
 - We stop this, when the subproblems are size 1, so will split the problem into subproblems $\log_2(n)$ times.

Analysis of merge sort

- The worst case running time of merge sort, $T(n)$ is:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

- To understand this equality, think “How many time did we recursively call the algorithm? ”
 - At each step, we make the subproblems half the size. So, at step i , the subproblems are each of size $n/(2^i)$.
 - We stop this, when the subproblems are size 1, so will split the problem into subproblems $\log_2(n)$ times.
 - Solving the subproblems, and merging to double in size the sorted subarrays, will take $\Theta(n)$ each time.

Analysis of merge sort

- The worst case running time of merge sort, $T(n)$ is:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log(n))$$

- To understand this equality, think “How many time did we recursively call the algorithm? ”
 - At each step, we make the subproblems half the size. So, at step i , the subproblems are each of size $n/(2^i)$.
 - We stop this, when the subproblems are size 1, so will split the problem into subproblems $\log_2(n)$ times.
 - Solving the subproblems, and merging to double in size the sorted subarrays, will take $\Theta(n)$ each time.
- Hence the total algorithm takes $T(n) = \Theta(n \log(n))$.

Homework (due with Assignment 1)

- Find a loop invariant for the first **while** loop of the MERGE pseudocode, and use it to show the algorithm is correct.
- Describe (with pseudocode) a recursive algorithm for insertion sort, which calls on itself to sort the sub-array $A[1:n-1]$ and the inserts $A[1]$